Competitive Programming

Section 40: Data Structures Mixed Contest

Competitive Programming Course

2026-10-05

Table of contents

1 Combining data-structure ideas 3

2 Four questions before choosing a structure 4
- 2.1 1. Is the processing online? . . . 4
- 2.2 2. What does the query return? . . . 4
- 2.3 3. What changes? . . . 4
- 2.4 4. Is the needed summary composable? . . . 4

3 Selection by descending a tree 5

4 Change the representation when updates are awkward 6

5 Logical deletion when a structure cannot update in place 7

6 Complexity must include the entire workflow 8

7 A disciplined contest workflow 9

8 Complete local contest set 10

9 Operation diagnosis, not an answer key 11

10 Invariants before code 12

11 Complexity checks 13

12 Exercises 14

1 Combining data-structure ideas

In a mixed contest, start from the operations, not the container name. A statement often hides a familiar operation behind a story: assigning a hotel is a search for the first feasible position; removing the k-th live element is an order-statistic query; counting values in a changing interval is maintaining a frequency invariant.

The transferable skill is translating prose into a small operation contract. For example:

```
state: an array of capacities
update: subtract x at one chosen position
query: first position whose value is at least x
```

Once written this way, compare the contract against structures you know. A Fenwick tree is excellent for invertible prefix information and prefix order statistics. A segment tree can store a general associative range summary and descend according to that summary. A heap exposes an extreme but does not support arbitrary range boundaries. Sometimes no single familiar structure supports the operations directly; changing the representation can turn one large update into a few point updates.

2 Four questions before choosing a structure

2.1 1. Is the processing online?

An online answer must be produced before later operations are known, or later updates depend on it. Offline queries may be reordered and then restored to their original output order. Reordering is powerful, but it is invalid when updates change the meaning of earlier and later queries unless time is also modeled explicitly.

2.2 2. What does the query return?

An **aggregate** combines a range: sum, minimum, maximum, or GCD. A **selection** asks for a position: first feasible, k-th active, or first prefix crossing a threshold. A **frequency query** counts values satisfying a condition. These require different invariants even when all mention ranges.

2.3 3. What changes?

Write the update exactly. Point assignment and point addition are different. Range addition changes many leaves but has a compact lazy representation. Toggling a position is a frequency change of plus or minus one. If nothing changes, a static or offline method may preprocess much more aggressively.

2.4 4. Is the needed summary composable?

A segment-tree node must summarize a segment without looking inside it. Sum, minimum, and maximum compose directly. Maximum subarray sum needs four fields, because the best answer may cross the child boundary. A proposed summary is sufficient only if two child summaries always determine the parent summary.

3 Selection by descending a tree

Many mixed-contest problems ask for an index rather than an aggregate. The central idea is to use a node summary to decide which child can contain the answer.

For a first position with value at least `x`, store the maximum in each node:

```
if left.maximum >= x: descend left
otherwise:            descend right
```

For the k-th active position, store counts:

```
if left.count >= k: descend left
otherwise: subtract left.count from k and descend right
```

At every level one child is discarded, so the search is `O(log n)`. The proof depends on the summary: discarding the left child is safe only because its maximum/count proves that the desired position cannot be there.

4 Change the representation when updates are awkward

Suppose an operation changes every value in `[l,r]`. Storing the values literally makes that update expensive. In the difference array, however, the same operation changes only two boundaries:

```
d[l]     += x
d[r + 1] -= x       if r + 1 exists
```

This does not automatically solve the query. It gives a new operation contract: point changes in `d`, reconstruction of one original value by a prefix sum, and perhaps a range aggregate over differences. Those pieces may belong in different structures.

The general lesson is to search for an identity that exposes the information needed by the query. Then assign each resulting operation to the smallest structure that supports it. A mixed solution is justified only after its invariants describe one consistent underlying array.

5 Logical deletion when a structure cannot update in place

A standard heap efficiently inserts and removes its top, but it cannot find an arbitrary old entry after that entry's priority changes. One useful pattern is to insert the new version and leave the old version in the heap. A separate table records the current version of every item.

Whenever the heap top is inspected, compare it with the table. If it is old, discard it and continue. This is **lazy deletion**: physical removal is delayed until the stale entry reaches the top.

The total cost remains logarithmic per operation because every heap entry is inserted once and removed at most once. The key invariant is that the table is authoritative; a heap entry is usable only when it agrees with that table.

6 Complexity must include the entire workflow

Suppose a query is `O(log n)`, but the code reconstructs the tree or sorts all values before every query. The real cost is still too large. Account for:

```
reading + compression + sorting + building + all updates + all queries
```

Also count memory. A nested structure that appears `O(n log n)` may have large constants; storing Python integers in millions of nodes can dominate runtime and memory even when the asymptotic bound is acceptable.

7 A disciplined contest workflow

For each task:

1. Translate the story into state, update, and query.
2. Reject the direct simulation using the constraints.
3. Write the invariant of the proposed structure in one sentence.
4. Trace one update and one query by hand.
5. Prove why every discarded region cannot contain the answer.
6. Calculate total time and memory.
7. Build a tiny brute-force oracle before optimizing tests.

A good tiny test is chosen to break an assumption: duplicate values for compression, negative values for prefix searches, touching interval endpoints, an empty active set, a full-range query, or repeated updates to the same position.

8 Complete local contest set

Problem	Title
A	Hotel Queries
B	List Removals
C	Range Add, Range GCD
D	Salary Queries
E	Functional Walk Minimum
F	Mutable Priority Queue
G	Streaming Room Count
H	Toggle K-th Active
I	Dynamic Brackets
J	Sliding Median Cost
K	Budget Prefix Search
L	Range Add Threshold Search

Problems E and F complete the direct category coverage. Problems G–L form a second, progressively harder half: each adds one modeling or invariant burden to structures already seen in Sections 36–39.

9 Operation diagnosis, not an answer key

For each local task, make an operation table:

Operation	Information needed	Online or reorderable?	Required cost
update	?	?	?
query	?	?	?

Then ask whether the answer depends on a prefix, a range aggregate, an order statistic, a first/last feasible position, or a temporary interval. The answer should determine the structure; do not select one just because it is familiar. The editorial gives the local reductions after an attempt.

10 Invariants before code

State exactly what every stored value represents. For a tree, specify the meaning of an internal node. For a frequency structure, specify which items are currently active. For offline processing, specify the current interval or set. After each update, explain why that statement remains true.

11 Complexity checks

Count operation cost over the whole input, not only one operation. A method with cheap additions can still be too slow if it rebuilds work for every query. If reordering is considered, also prove that output order can be restored and that operations commute with the reordering.

12 Exercises

Attempt A through L without looking for a named technique. For each, record the operation table, your invariant, and one naive method rejected by the constraints. Treat A–F as the core and G–L as the progressive challenge ladder. Use the editorial only after that record is complete.